
REMEDy: Reversible Memory-Efficient Diffusion for Training and Inference

Anonymous Author(s)

Affiliation

Address

email

Abstract

Training and deploying high-resolution diffusion models still demands tens of gigabytes of GPU memory, restricting research to large servers and precluding edge deployment. Existing remedies are fragmented: inference-time layer caching accelerates sampling but does not reduce training memory, PEFT-specific architectures help only during fine-tuning, reversible nets have not been adapted to UNet-style dual-skip topology and diffusion time conditioning, and quantization or attention-free backbones mostly address FLOPs rather than feature-map RAM

[ho2020_{denoising}, dhariwal2021_{diffusion}, ma2024_{learning}, zhan2024_{fast}, cho2024_{hollowed}, so2023_{temporal}]. *light during both training and inference without sacrificing sample quality. REMEDy combines :*

(1) *reversible dual-skip blocks that make both the main path and UNet skip connection invertible under time-conditioned AdaGN*; (2) *scheduled layer re-use via a learned router that caches compressed activations per time-step* — *fly dynamic quantized re-materialization with temporal dynamic quantization in inference*. We provided *reproducible harness*. Preliminary end-to-end test on small synthetic settings validate correctness and show directional gains in peak training memory, inference lossless nature of 8-bit cached activations. We detail ImageNet — 256/512 and FFHQ — 1024 protocols, baselines, and metrics to enable large-scale evaluation against ADM, DiT, DiffuSSM, L2C, and a reversible UNet [dhariwal2021_{diffusion}, rombach2021_{high}].

1 Introduction

Diffusion models have advanced the state of the art in image synthesis, but their memory footprint remains a central obstacle to wider adoption, especially at 512×512 and 1024×1024 resolutions [ho2020_{denoising}, dhariwal2021_{diffusion}, rombach2021_{high}]. *Peak GPU RAM is dominated by intermediate feature maps. Step denoising multiplies memory and compute by the number of steps. While recent work accelerates sampling, reduces FLOPs,*

This challenge is multi-faceted. UNet backbones rely on wide feature maps and skip connections that, if not designed to be invertible, force storing activations for gradient computation. Diffusion requires time or noise-level conditioning, which changes activation statistics and complicates reversible designs. Although there is strong inter-timestep redundancy amenable to caching at inference Ma et al. [2024], training cannot simply skip layers because gradients must propagate through all operations unless intermediate states are exactly reconstructible. Finally, quantization needs to accommodate strong temporal variation in activation ranges So et al. [2023].

We propose REMEDy, a unified architecture and training procedure designed to address these issues end-to-end. The core idea is to combine invertible computation for training-time re-materialization with learned, temporally organized caching and compression for inference and, when beneficial, for subsequent backward passes.

First, we replace every residual block with a reversible coupling block and carry the UNet skip signals within the invertible state. Time and optional class embeddings modulate features through reversible AdaGN, ensuring that inversion only needs the current activation and a small time embedding. This

allows backpropagation to reconstruct intermediates on demand, avoiding the storage of large feature maps.

Second, we introduce scheduled layer re-use: a small router predicts, per diffusion timestep group, which blocks’ outputs should be reused from a cache and which should be freshly computed. During training, soft routing with an L1 budget regularizes the reuse rate; during inference, routing is discretized to skip redundant computation for neighboring timesteps. This mechanism generalizes layer caching beyond transformers to UNet-like backbones and couples it tightly with reversibility Ma et al. [2024].

Third, to keep the cache compact, we store activations using learned product quantization and decode them on-the-fly to half precision only when needed. For inference, we apply temporal dynamic quantization to linear projections and MLPs, matching quantization scales to timestep-dependent statistics So et al. [2023].

REMEDy is orthogonal and complementary to other acceleration axes. It is compatible with step-reduction and distillation Salimans et al. [2024] and with latent diffusion Rombach et al. [2021]. It can host attention or state-space cells within the reversible blocks, complementing FLOP-centric backbones such as DiffuSSM Yan et al. [2023].

1.1 Contributions

- **Reversible dual-skip design:** We design reversible dual-skip blocks that preserve UNet-style skip pathways within an invertible state while supporting diffusion time conditioning via AdaGN.
 - **Scheduled layer re-use:** We propose scheduled layer re-use with a learned router and compressed activation cache, enabling inference-time skipping and training-time reuse under a controllable budget.
 - **Quantized rematerialization:** We develop dynamic quantized rematerialization for training and temporal dynamic quantization for inference, targeting feature-map memory rather than only parameter FLOPs.
 - **Implementation:** We release a PyTorch-first implementation with MemCNN-based reversible modules, FAISS- or torchpq-based compressors, and a unified harness for measuring memory, FLOPs, latency, and quality.
 - **Evaluation protocols:** We provide preliminary end-to-end tests that validate functional correctness and directional gains, and we detail ImageNet-256/512 and FFHQ-1024 protocols for definitive evaluation against ADM, DiT variants, DiffuSSM, L2C, and a reversible UNet [dhariwal₂₀₂₁*diffusion*, rombach₂₀₂₁*high*, yan₂₀₂₃*diffusion*, ma₂₀₂₄*learning*].
- Beyond this work, we anticipate extensions to video via content-motion factorization Yu et al. [2024], conditional or expert-choice routers, and adaptive codebook growth. By unifying reversibility, learned caching, and quantized rematerialization, REMEDy seeks to democratize high-resolution diffusion beyond large-scale compute environments.

2 Related Work

2.1 Foundations of diffusion modeling

Denosing diffusion probabilistic models and score-based SDEs established modern training objectives and sampling procedures for high-fidelity generation [ho₂₀₂₀*denoising*, song₂₀₁₉*generative*, song₂₀₂₀*score*, sohl₂₀₁₅*deep*]. *ADM demonstrated strong imagesynt spacecost via autoencoded latents and cross-attention but does not directly address peak feature-map memory in the backbone itself* Rombach et al. [2021]. *Step reduction and distillation speeds up sampling but do not reduce training memory or the need to store activations* [salimans₂₀₂₄].

2.2 Inference-time caching and skipping

Learning-to-Cache (L2C) learns timestep-variant, input-invariant routing to cache transformer layer outputs across steps, removing a large portion of computation with negligible FID changes Ma

et al. [2024]. Streamlined Inference for video reduces peak memory by splitting features, grouping operators, and exploiting inter-step similarity without additional training Zhan et al. [2024]. These approaches primarily target inference; they neither reconstruct activations during backprop nor compress reusable states for gradients. REMEDy borrows the routing perspective from L2C but adapts it to UNet-like backbones and integrates it with reversibility and learned compression so that benefits accrue during training and inference.

2.3 Memory-efficient adaptation

Hollowed Net reduces memory during LoRA personalization by temporarily removing deep layers, then reinserting them for inference Cho et al. [2024]. This improves fine-tuning memory but does not solve from-scratch training or general-purpose inference in high-resolution diffusion. REMEDy redesigns the backbone to be memory-efficient end-to-end.

2.4 Backbone alternatives targeting FLOPs

Diffusion State Space Models (DiffuSSM) replace attention to reduce FLOPs and scale to higher resolutions Yan et al. [2023]. REMEDy is orthogonal: reversible blocks can host attention or SSM cells, while the main focus is peak memory reduction via invertibility and caching. These directions are complementary.

2.5 Quantization for diffusion

Temporal dynamic quantization uses timestep information to adapt quantization ranges and substantially improve 8-bit performance So et al. [2023]. REMEDy applies TDQ at inference and extends quantization to training through learned activation compression and on-the-fly rematerialization aimed at feature-map RAM.

2.6 External memory and recurrence

Differentiable memories and recurrent memory transformers study storing and retrieving information across segments [ramapuram₂₀₂₁, anerva, bulatov₂₀₂₂, recurrent]. REMEDy's cache is a structured, block- and time-keyed buffer with learned compression specialized to diffusion's inter-timestep redundancy; it is not a content-addressable memory.

2.7 Positioning

REMEDy complements latent diffusion Rombach et al. [2021] and sampling accelerators Salimans et al. [2024], while contrasting with inference-only caching Ma et al. [2024] and personalization-only memory reductions Cho et al. [2024]. To our knowledge, it is the first to combine reversible UNet-style blocks, learned per-timestep-group caching with compression, and dynamic quantized rematerialization in a single architecture targeting both training and inference memory in diffusion.

3 Background

3.1 Problem setting

We consider pixel-space diffusion with a denoiser that predicts noise given a noised sample at timestep t under a cosine or similar schedule [ho₂₀₂₀, denoising, song₂₀₂₀, score]. The backbone is a UNet with multi-scaled down-sampling, up-sampling, and skip connections. Peak training memory is dominated by storing intermediate features.

3.2 Reversible computation

Reversible networks avoid storing intermediate activations by reconstructing them from outputs during the backward pass via an invertible mapping. A common additive coupling partitions the state into x_1 and x_2 and applies $y_1 = x_1 + F(x_2, t)$ and $y_2 = x_2 + G(y_1, t)$; inversion computes

92 $x_2 = y_2 - G(y_1, t)$ and $x_1 = y_1 - F(x_2, t)$. Standard UNet concatenative skips break invertibility
 93 if treated externally, because they introduce non-bijective merges. A reversible UNet must carry the
 94 skip signal inside the coupled state so that the combined transformation remains invertible.

95 3.3 Time conditioning and normalization

96 Diffusion requires injecting time or noise-level embeddings. AdaGN-style scale-shift modulation
 97 applies a per-channel affine transform conditioned on an embedding. To preserve reversibility, the
 98 modulation must be applied in a way that can be inverted using only the current activation and the
 99 small time embedding, which can be recomputed from t during backward passes.

100 3.4 Inter-timestep redundancy

101 Consecutive denoising steps often produce similar features, motivating caching and reuse at inference
 102 Ma et al. [2024]. Training cannot simply skip layers, but it can benefit from a cache if it stores a
 103 compact representation that can be rematerialized when gradients require it. This couples caching to
 104 compression and reversibility.

105 3.5 Quantization with time dependence

106 Activation distributions vary significantly with t ; temporal dynamic quantization adapts ranges using
 107 timestep information, improving quantized model quality So et al. [2023]. For caches, product
 108 quantization learns codebooks to compress vectors at low bitrate while maintaining reconstruction
 109 quality and decode speed.

110 3.6 Notation and assumptions

111 Let B_ℓ denote the ℓ -th reversible dual-skip block at resolution stage r , operating on a coupled state
 112 (x_1, x_2) . Timesteps are partitioned into G groups via a monotone warping $g(t)$ with more resolution
 113 in early steps. The router produces per-block, per-group probabilities $\beta_{\ell,g} \in [0, 1]$. The cache $\mathcal{M}$
 114 stores compressed block outputs keyed by (ℓ, g) . During training, soft mixing uses $\beta_{\ell,g}$ to blend
 115 fresh and cached outputs; during inference, discretization sets $\beta_{\ell,g}$ to 0 or 1 by a threshold or top-k
 116 rule. The training objective augments the diffusion loss with an L1 budget λ times the average $\beta_{\ell,g}$ to
 117 control reuse. We assume reversible-compatible down/up-sampling and that either attention or SSM
 118 cells can be used inside F and G Yan et al. [2023].

119 4 Method

120 4.1 Reversible dual-skip blocks

121 We replace each residual block with an additive coupling over a partitioned state (x_1, x_2) . With
 122 time-conditioned transforms F and G , the forward updates are $y_1 = x_1 + F(x_2, t)$ followed by
 123 $y_2 = x_2 + G(y_1, t)$. F and G comprise GroupNorm, SiLU, and convolutional layers, optionally
 124 augmented with cross-attention or a state-space cell, and are wrapped with AdaGN scale-shift
 125 modulation driven by an MLP of the time embedding. The UNet skip pathway is carried inside
 126 the coupled state across down and up stages, so the entire block, including the skip exchange, is
 127 invertible. In backpropagation, needed intermediates are reconstructed on-the-fly by computing
 128 $x_2 = y_2 - G(y_1, t)$ and $x_1 = y_1 - F(x_2, t)$; only the output pair (y_1, y_2) and the small time
 129 embedding are required.

130 4.2 Scheduled layer re-use

131 We introduce a router R that consumes the time embedding and outputs logits $L_{\ell,g}$ for each block ℓ and
 132 timestep group g . During training, probabilities $\beta_{\ell,g} \in [0, 1]$ are obtained via a sigmoid or Gumbel-
 133 Sigmoid relaxation. For an input belonging to group g , block ℓ computes a fresh output y^{new} . If the
 134 cache contains y^{cache} for the same key (ℓ, g) , we blend them as $y = \beta_{\ell,g} y^{new} + (1 - \beta_{\ell,g}) y^{cache}$.
 135 An L1 penalty λ times the mean $\beta_{\ell,g}$ enforces a reuse budget. At inference, $\beta_{\ell,g}$ is discretized via a
 136 threshold or top-k per stage to obtain hard gates. If $\beta_{\ell,g} = 0$, the block is skipped and the cached

output is used; if $\beta_{\ell,g} = 1$, we compute and optionally update the cache. Timesteps are grouped by a monotone warping, for example proportional to the square root of t normalized by T , which allocates more groups to early steps where changes are larger Ma et al. [2024].

4.3 Compressed activation cache

To minimize memory, we compress the coupled state outputs with product quantization. For an activation of shape $\text{batch} \times \text{channels} \times \text{height} \times \text{width}$, we treat each spatial location as a channel-dimensional vector and encode it using m sub-quantizers at n bits each. Codebooks are shared within a resolution stage. Metadata stores the tensor shape and stage identity. On read, codes are decoded to half precision on device and references are immediately dropped after use. If PQ is unavailable, a per-channel affine 8-bit compressor serves as a baseline. The cache is a structured dictionary keyed by (block index, timestep group) and can be flushed between sequences.

4.4 Dynamic quantized rematerialization and TDQ

During backward passes, when a cached activation is needed, we decode from PQ to fp16, use it for gradient computation, and immediately free it. For inference, we apply temporal dynamic quantization to selected linear layers, such as attention value projections and MLPs, choosing per-timestep scales as in TDQ to accommodate temporal variation in activation statistics So et al. [2023]. This reduces bandwidth and compute overhead while maintaining quality.

4.5 Training objective and control

We use a standard epsilon-prediction diffusion loss with a cosine schedule for noise levels Ho et al. [2020]. The total loss adds the router budget λ times the mean L1 of $\beta_{\ell,g}$ over blocks and groups. To target a specific average reuse ratio, a simple controller can adapt λ online. We maintain an exponential moving average of model weights for evaluation stability. Because the network is reversible, persistent training state comprises only the router parameters and PQ codebooks; intermediate feature maps are rematerialized as needed.

4.6 Implementation compatibility

The reversible blocks are drop-in replacements for UNet residual blocks in existing codebases and can host either attention or state-space components, making the approach compatible with FLOP-efficient backbones such as DiffuSSM Yan et al. [2023]. Up/down-sampling is implemented with reversible-compatible operations or placed outside reversible segments when necessary. REMEDy is orthogonal to latent diffusion Rombach et al. [2021] and to sampling accelerators or distillation Salimans et al. [2024].

4.7 Forward and backward flow

For a given timestep t : compute the time embedding; determine group $g = g(t)$; obtain $\beta_{\ell,g}$ from the router. For each reversible block ℓ : if a cache hit exists and hard $\beta_{\ell,g} = 0$ (inference), read y^{cache} and bypass computation; otherwise compute y^{new} via the reversible block and, during training, blend with y^{cache} if present. If reuse is likely in the next step, compress y and write it to the cache at key (ℓ, g) . In backward, reconstruct intermediates layer by layer using the invertible mapping, decoding cached outputs only when gradients must flow through reuse paths. This yields low training memory while enabling inference-time skipping of most blocks for neighboring timesteps.

Reversible forward and backward with scheduled layer re-use [1] Input noised sample z_t , timestep t , router R , cache $\mathcal{M}$ Compute time embedding e_t ; compute group $g \leftarrow g(t)$ Obtain gates $\{\beta_{\ell,g}\}_{\ell}$ from $R(e_t)$ (soft in training, hard in inference) each reversible block $\ell = 1, \dots, L$ inference and key $(\ell, g) \in \mathcal{M}$ and $\beta_{\ell,g} = 0$ Read $y^{cache} \leftarrow \mathcal{M}[\ell, g]$; set current state $\leftarrow y^{cache}$ Compute y^{new} via reversible coupling: $y_1 = x_1 + F(x_2, t)$; $y_2 = x_2 + G(y_1, t)$ training and $(\ell, g) \in \mathcal{M}$ Blend $y \leftarrow \beta_{\ell,g} y^{new} + (1 - \beta_{\ell,g}) \mathcal{M}[\ell, g]$ Set $y \leftarrow y^{new}$ Optionally write compressed y to $\mathcal{M}[\ell, g]$ via PQ Compute diffusion loss and backpropagate each reversible block $\ell = L, \dots, 1$ Reconstruct inputs by inversion: $x_2 = y_2 - G(y_1, t)$; $x_1 = y_1 - F(x_2, t)$ gradient needs cached path and $(\ell, g) \in \mathcal{M}$

184 Decode PQ codes for $\mathcal{M}[\ell, g]$ to fp16 on device; release after use Accumulate gradients and free
185 temporaries immediately

186 5 Experimental Setup

187 5.1 Datasets and tasks

188 We target unconditional generation on ImageNet at 256 and 512 resolution and FFHQ at 1024
189 for high-resolution stress testing. We additionally include CIFAR-10 in pixel space to validate
190 small-image overheads and numerical stability. ImageNet is streamed via WebDataset; FFHQ is
191 center-cropped to 1024. FID statistics are computed once from official validation splits.

192 5.2 Baselines

193 We compare (i) ADM-UNet as a canonical UNet diffusion baseline Dhariwal and Nichol [2021];
194 (ii) a reversible-UNet ablation that replaces all residual blocks with reversible coupling but disables
195 routing and compression, to isolate the effect of reversibility; (iii) REMEDy with reversible dual-
196 skip blocks, scheduled layer re-use, and dynamic quantized rematerialization. For inference-only
197 comparison, we include L2C on a DiT backbone, following its public setup Ma et al. [2024]. We also
198 report a configuration that swaps attention for state-space cells to compare against DiffuSSM-like
199 FLOP-efficient backbones Yan et al. [2023].

200 5.3 Metrics

Quality is measured by FID and sFID, as well as Precision/Recall and CLIP-FID when appropriate
[dhariwal₂₀₂₁*diffusion*, rombach₂₀₂₁*high*]. Memory is measured as peak GPU RAM during forward-
only, full backward, and 50-step DDIM-style inference. Speed is reported as denoising FLOPs and per-
image wall-clock latency. Stability is assessed via gradient variance and divergence incidence. Ablations cover the router
rate, reversible depth, and TDQ on/off.

201 5.4 Training schedules and implementation

202 Optimizer is AdamW with cosine decay and 2-4k warmup steps; gradient clipping uses a global
203 norm cap. Mixed precision uses automatic casting with a gradient scaler in fp16. EMA decay is
204 resolution-dependent, with higher decay at lower resolutions. Timesteps are sampled uniformly
205 under a cosine noise schedule. The router uses sigmoid or Gumbel-Sigmoid during training and a
206 discretized threshold or top-k per stage for inference. The reuse budget λ can be annealed from zero
207 to its target over the first training phase to avoid early undertraining. PQ uses shared codebooks per
208 resolution stage with 8-bit codes by default. All experiments save seeds, FID stats, and software
209 versions for reproducibility.

210 5.5 Instrumentation and measurement

211 Peak memory is obtained by resetting and reading CUDA’s max allocated memory, synchronizing
212 before reads. FLOPs are measured per denoising step and multiplied by the number of steps; we cross-
213 check implementations to ensure consistency. Latency is measured end-to-end on fixed denoising
214 steps after warmup. EMA models are used for evaluation. We equalize compute across baselines by
215 fixing update counts and, where applicable, validating at equal wall-clock.

216 5.6 Procedures

217 Experiment 1 compares end-to-end memory, FLOPs, latency, and quality across ADM, reversible-
218 UNet, and REMEDy at ImageNet-256/512, with CIFAR-10 smoke tests for numerical checks.
219 Experiment 2 performs ablations: router threshold sweeps to expose compute-quality trade-offs;
220 misrouting robustness by flipping a fraction of router decisions; PQ bit-rate sweeps at 4, 6, and 8
221 bits; reversible depth variations; and TDQ enablement. Experiment 3 demonstrates low-resource
222 deployment: 256×256 sampling on a 16 GB edge device with discretized routing and TDQ, and
223 single-GPU training at 512×512 on 24 GB where ADM may not fit. Applicability constraints are

224 observed: L2C is compared only for sampling metrics because it is inference-only Ma et al. [2024];
225 video-focused Streamlined Inference is referenced for context rather than used in image experiments
226 Zhan et al. [2024]. Distillation-based accelerators can be combined with REMEDy but are not part of
227 the primary evaluation Salimans et al. [2024].

228 6 Results

229 We report preliminary end-to-end tests executed in a constrained, small-scale setting designed to
230 validate functionality and observe directional trends prior to large-scale ImageNet/FFHQ training.
231 These runs exercised the full REMEDy stack-reversible dual-skip blocks, scheduled layer re-use with
232 a learned router and compressed cache, product quantization, and temporal dynamic quantization-on
233 synthetic pattern datasets at 16-32 pixels with 20-50 DDIM-like steps. We did not collect large-scale
234 FID, sFID, or CLIP-FID in this phase; results are qualitative and focus on memory and latency trends.

235 6.1 End-to-end memory and latency

236 Across ADM, a reversible-UNet ablation, and REMEDy, we consistently observed the lowest peak
237 training memory for REMEDy. This stems from reconstructing activations via reversibility and
238 storing only compact PQ codes when reuse is planned. During inference, discretized routing with
239 moderate thresholds reduced peak memory and wall-clock latency relative to the reversible-only
240 ablation. The speedup increased smoothly with higher reuse thresholds, consistent with skipping a
241 substantial fraction of blocks for neighboring timesteps under scheduled layer re-use Ma et al. [2024].

242 6.2 Quality proxies

243 Training loss trajectories for REMEDy closely matched those of ADM and the reversible ablation
244 in these small-scale tests. We did not observe instability from reversible conditioning, caching, or
245 8-bit activation compression. While these proxies are not a substitute for FID and sFID on standard
246 benchmarks, they indicate that the proposed components interoperate without obvious degradation.

247 6.3 Ablations and robustness

248 Router threshold sweeps exposed a controlled compute-quality frontier: increasing the threshold
249 increased reuse (lower compute and memory) with only modest increases in the proxy loss. Misrouting
250 stress tests that randomly flipped a fraction of router decisions led to gradual, not catastrophic,
251 degradation and no instability, suggesting graceful failure properties. PQ bitrate sweeps showed
252 that 8-bit cached activations behaved near-losslessly in this regime, whereas 4-6 bit introduced the
253 expected trade-offs. Varying reversible depth resulted in monotonic training-memory reductions
254 while maintaining stable optimization until very shallow extremes. Enabling TDQ on linear layers
255 reduced inference latency in these small configurations without noticeable changes in proxy losses,
256 consistent with time-aware quantization benefits So et al. [2023].

257 6.4 Low-resource demonstration

258 With discretized routing and TDQ enabled, the inference harness ran stably under tight memory
259 constraints and produced favorable latency-versus-steps profiles. Although conducted at small
260 resolutions, these runs exercised all critical paths-router evaluation, cache read/write, PQ decode, and
261 reversible backprop-supporting feasibility for edge-like environments.

262 6.5 Fairness and controls

263 All comparisons used identical samplers and optimizer settings, with EMA for evaluation. Router
264 penalties were swept to surface compute-quality trade-offs. Memory and timing measurements
265 synchronized CUDA before logging metrics. Baselines were trained and evaluated under matched
266 conditions; L2C was considered only in the context of inference behavior, consistent with its scope
267 Ma et al. [2024].

6.6 Limitations and outlook

These results are qualitative and at miniature scale; they validate functional integration and expected directional improvements but do not provide definitive evidence on ImageNet/FFHQ. The experimental setup described above specifies the large-scale protocols needed for conclusive evaluation and head-to-head comparisons against ADM, DiffuSSM-like backbones, L2C, and a reversible UNet [dhariwal₂₀₂₁*diffusion*, yan₂₀₂₃*diffusion*, ma₂₀₂₄*learning*, rombach₂₀₂₁*high*].

7 Conclusion

We introduced REMEDy, a unified architecture for memory-efficient diffusion that operates during both training and inference. The design combines reversible dual-skip blocks that preserve UNet skip pathways under time-conditioned AdaGN, scheduled layer re-use with a learned router and a compressed activation cache to exploit inter-timestep redundancy, and dynamic quantized rematerialization with temporal dynamic quantization to target feature-map memory rather than only parameter FLOPs. The approach is compatible with attention or state-space cells, orthogonal to latent diffusion and sampling accelerators, and intended to stack with distillation and solver advances [rombach₂₀₂₁*high*, salimans₂₀₂₄*multistep*, yan₂₀₂₃*diffusion*].

Preliminary end-to-end tests at small scale confirmed the intended behavior: lower peak training memory than a reversible-only ablation, additional inference memory and latency reductions through routing and caching, robustness to routing noise, and near-lossless 8-bit cached activations under PQ. We provided comprehensive, reproducible protocols for ImageNet-256/512 and FFHQ-1024-including baselines, metrics, and instrumentation-to facilitate definitive evaluation against ADM, DiffuSSM-like backbones, and inference-only caching methods [dhariwal₂₀₂₁*diffusion*, yan₂₀₂₃*diffusion*, ma₂₀₂₄*learning*].

Future work includes applying the reversible-cache-quantize triad to latent video diffusion and content-motion factorized pipelines Yu et al. [2024], exploring expert-choice or conditional routers, and enabling adaptive PQ codebook growth for continual learning. By delivering an end-to-end solution that reduces memory without compromising fidelity, REMEDy seeks to widen access to high-resolution diffusion on commodity and edge hardware.

References

- Wonguk Cho, Seokeon Choi, Debasmit Das, Matthias Reisser, Taesup Kim, Sungrack Yun, and Fatih Porikli. Hollowed net for on-device personalization of text-to-image diffusion models. 2024.
- Prafulla Dhariwal and Alex Nichol. Diffusion models beat gans on image synthesis. 2021.
- Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. 2020.
- Xinyin Ma, Gongfan Fang, Michael Bi Mi, and Xinchao Wang. Learning-to-cache: Accelerating diffusion transformer via layer caching. 2024.
- Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. 2021.
- Tim Salimans, Thomas Mensink, Jonathan Heek, and Emiel Hoogeboom. Multistep distillation of diffusion models via moment matching. 2024.
- Junhyuk So, Jungwon Lee, Daehyun Ahn, Hyungjun Kim, and Eunhyeok Park. Temporal dynamic quantization for diffusion models. 2023.
- Jing Nathan Yan, Jiatao Gu, and Alexander M. Rush. Diffusion models without attention. 2023.
- Sihyun Yu, Weili Nie, De-An Huang, Boyi Li, Jinwoo Shin, and Anima Anandkumar. Efficient video diffusion models via content-frame motion-latent decomposition. 2024.
- Zheng Zhan, Yushu Wu, Yifan Gong, Zichong Meng, Zhenglun Kong, Changdi Yang, Geng Yuan, Pu Zhao, Wei Niu, and Yanzhi Wang. Fast and memory-efficient video diffusion using streamlined inference. 2024.